

SERVERLESS E-COMMERCE ORDER PROCESSING SYSTEM USING AWS

Project Portfolio

Prepared By:

Rogith Zen MSB

Technologies Used:

- **Amazon S3**
- **Amazon API Gateway**
- **AWS Lambda**
- **Amazon DynamoDB**
- **Amazon SQS**
- **Amazon SNS**

Role: Cloud & DevOps Learner

Date: 8June 2026

INDEX

1. Project Overview

2. Business Problem

3. AWS Services Used

4. Workflow Overview

5. Project Implementation

6. API Gateway Configuration

7. Lambda Function 1 – CreateOrder

8. DynamoDB Order Storage

9. Amazon SQS Integration

10. Lambda Function 2 – ProcessOrder

11. Amazon SNS Notification Service

12. Email Notification

13. Challenges Encountered

14. Key Outcomes

15. Skills Demonstrated

16. Conclusion

1.Project Overview

This project demonstrates a fully serverless e-commerce order processing application built using Amazon Web Services (AWS). The system allows users to place orders through a web interface hosted on Amazon S3. The orders are processed through an event-driven architecture using Amazon API Gateway, AWS Lambda, Amazon DynamoDB, Amazon SQS, and Amazon SNS.

The solution eliminates the need for server management while providing scalability, reliability, and automated order processing capabilities.

2.Business Problem

Traditional e-commerce applications require dedicated servers, infrastructure management, and manual scaling. As the number of users grows, maintaining performance and reliability becomes more challenging.

The objective of this project is to build a serverless order processing system that automatically scales, stores customer orders, processes them asynchronously, and sends automated notifications without managing any servers.

3.AWS Services Used

Amazon S3

Used to host the frontend website using Static Website Hosting.

Amazon API Gateway

Used to create REST API endpoints for receiving customer orders.

AWS Lambda

Used to execute backend business logic without managing servers.

Amazon DynamoDB

Used as a NoSQL database to store customer order details.

Amazon SQS (Simple Queue Service)

Used to queue order messages for asynchronous processing.

Amazon SNS (Simple Notification Service)

Used to send email notifications after successful order processing.

4.Workflow Overview

The application follows a serverless event-driven architecture.

Workflow:

1. The user accesses the website hosted on Amazon S3.
2. The user enters order details and submits the form.
3. Amazon API Gateway receives the request.
4. API Gateway invokes Lambda Function 1 (CreateOrder).
5. Lambda Function 1 stores the order in DynamoDB.
6. Lambda Function 1 sends the order information to Amazon SQS.
7. Amazon SQS triggers Lambda Function 2 (ProcessOrder).
8. Lambda Function 2 processes the order and publishes a message to Amazon SNS.
9. Amazon SNS sends an email notification to subscribed users.

Application Flow:

Frontend Website (S3)

↓

Amazon API Gateway

↓

Lambda Function 1 (CreateOrder)

↓

Amazon DynamoDB

↓

Amazon SQS

↓

Lambda Function 2 (ProcessOrder)

↓

Amazon SNS



Email Notification

5. Project Implementation

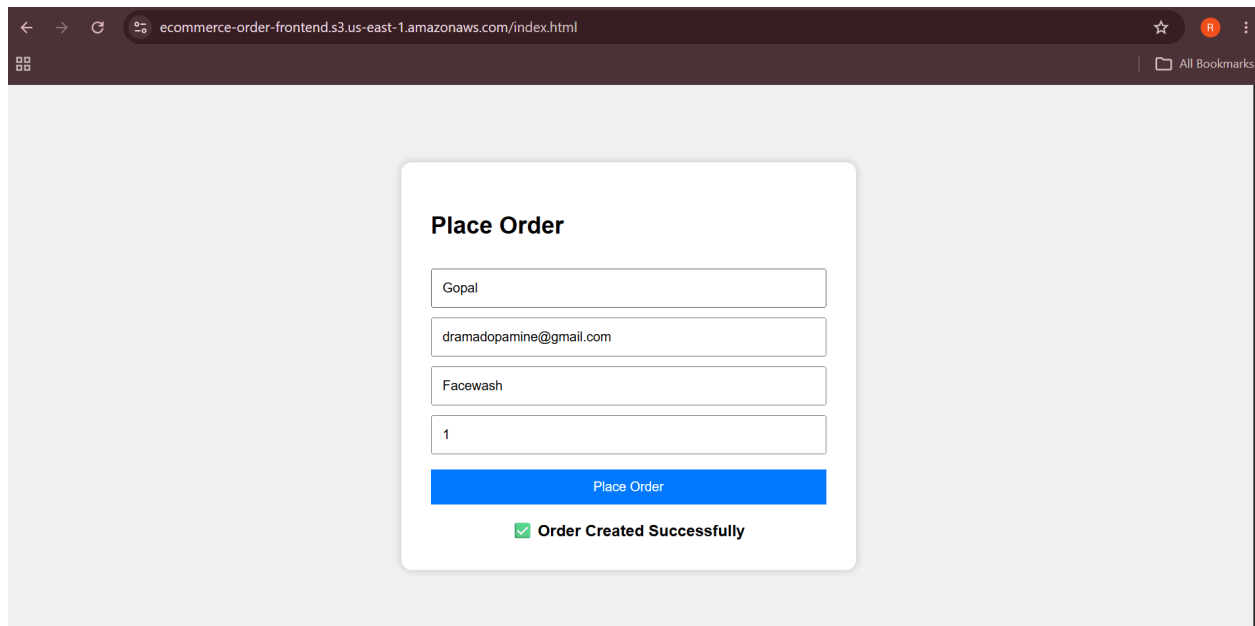
Frontend Application

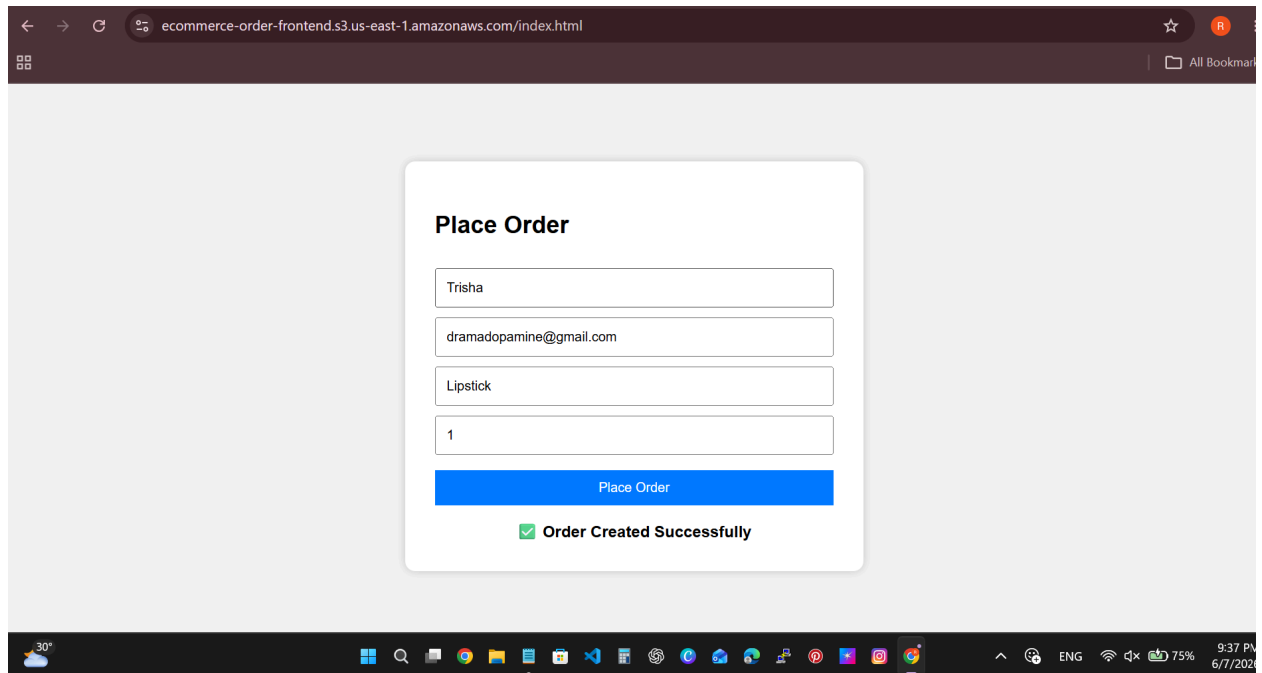
A responsive frontend application was developed using HTML, CSS, and JavaScript.

Features:

- Customer order form
- Product information entry
- Quantity selection
- API integration using JavaScript Fetch API
- Responsive user interface

The frontend is hosted on Amazon S3 using Static Website Hosting.





6.API Gateway Configuration

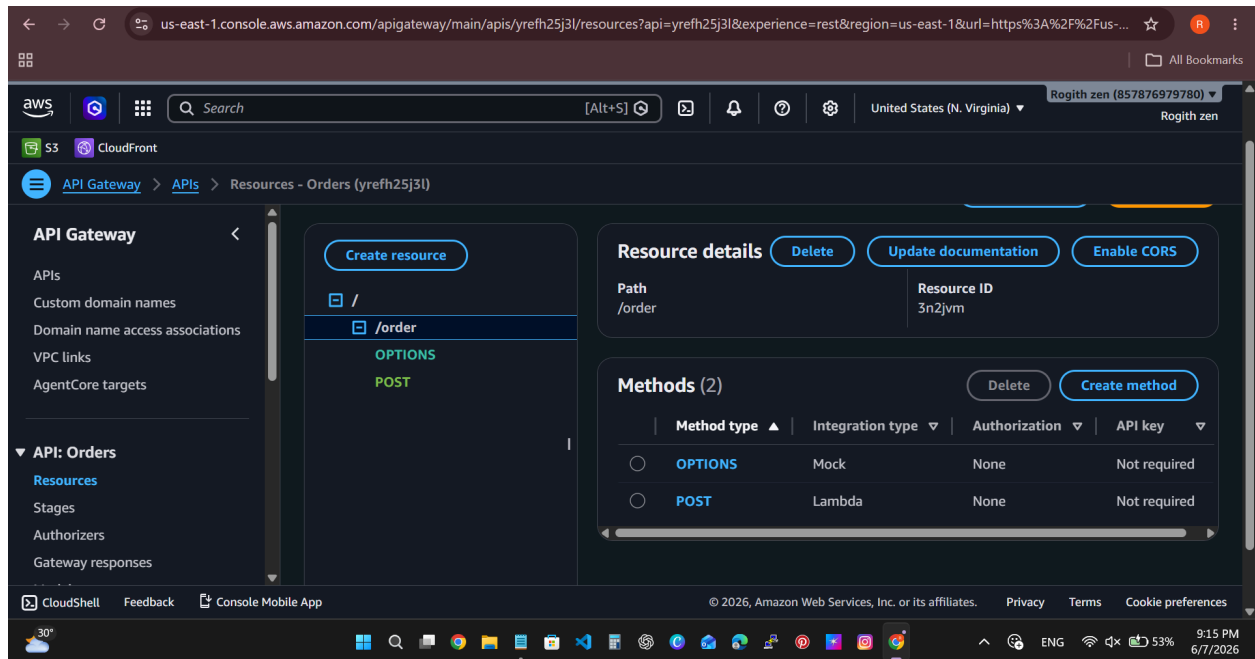
Amazon API Gateway was used to create a REST API endpoint for receiving customer order requests.

Endpoint:

POST /order

Responsibilities:

- Receive customer requests
- Route requests to AWS Lambda
- Enable communication between frontend and backend
- Support CORS for browser-based access



7.Lambda Function 1 – CreateOrder

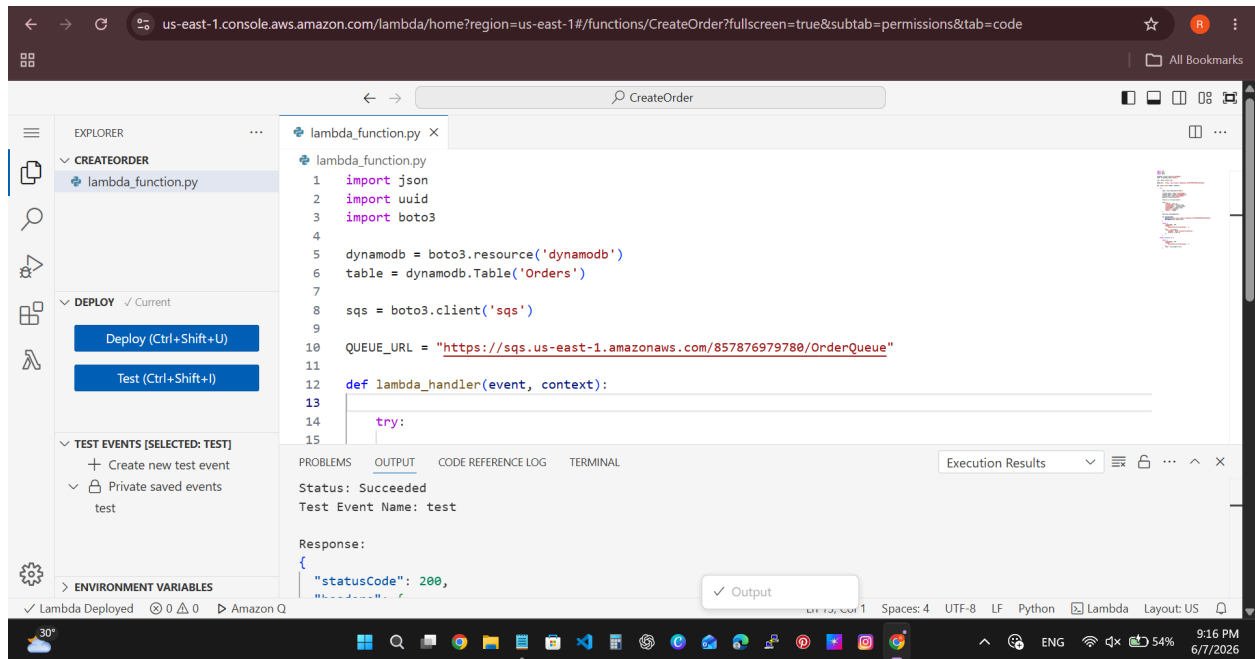
The CreateOrder Lambda function is responsible for handling incoming customer orders.

Main Responsibilities:

- Receive order details from API Gateway
- Generate a unique Order ID
- Store order details in DynamoDB
- Send order information to Amazon SQS

Benefits:

- Fast order creation
- Reliable order storage
- Decoupled architecture



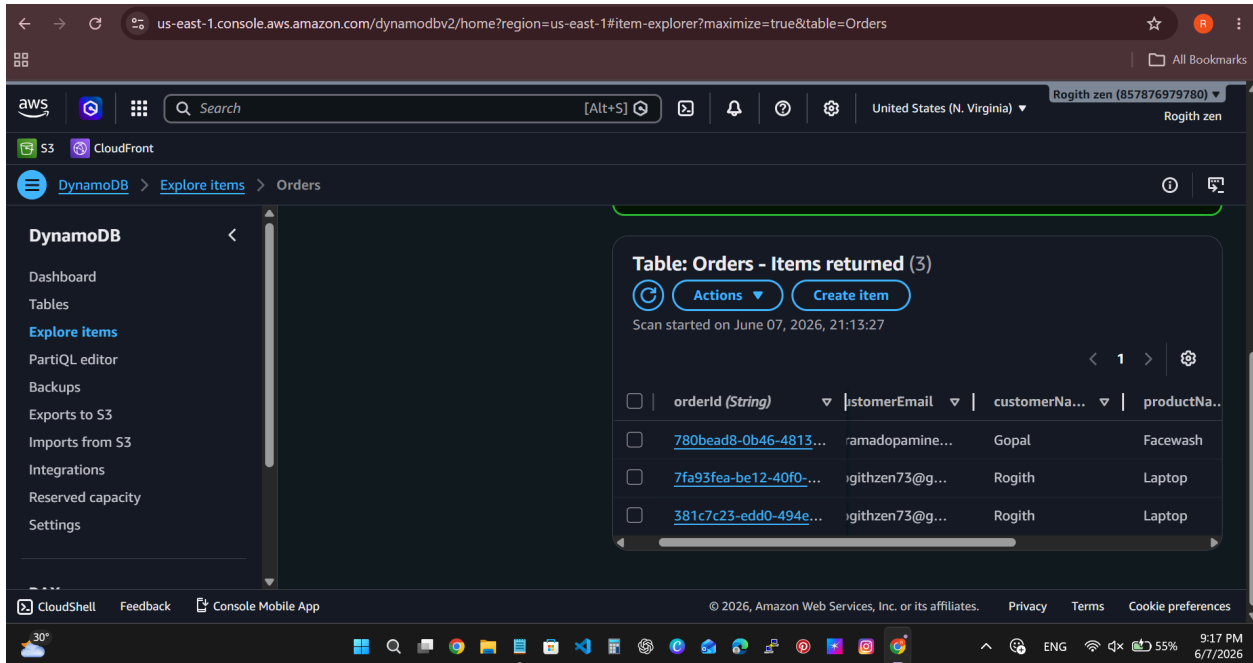
8.DynamoDB Order Storage

Amazon DynamoDB is used to store customer order information.

Stored Data:

- Order ID
- Customer Name
- Customer Email
- Product Name
- Quantity
- Order Status

DynamoDB provides high availability, scalability, and low-latency performance.



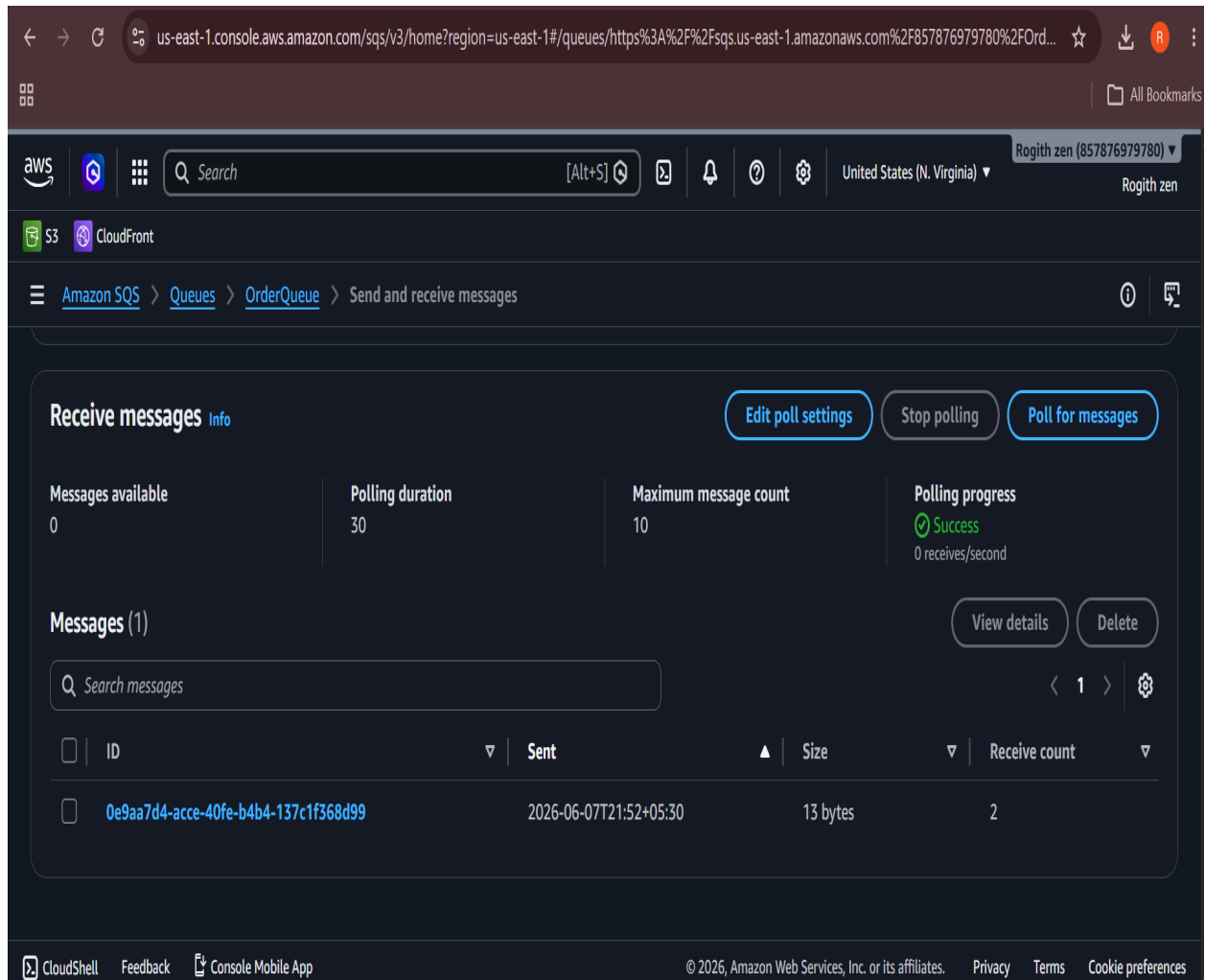
9. Amazon SQS Integration

Amazon SQS is used to queue order messages.

Purpose:

- Decouple application components
- Improve reliability
- Enable asynchronous processing
- Handle large volumes of orders

When an order is created, Lambda Function 1 sends a message to the SQS queue. The message remains in the queue until it is processed.



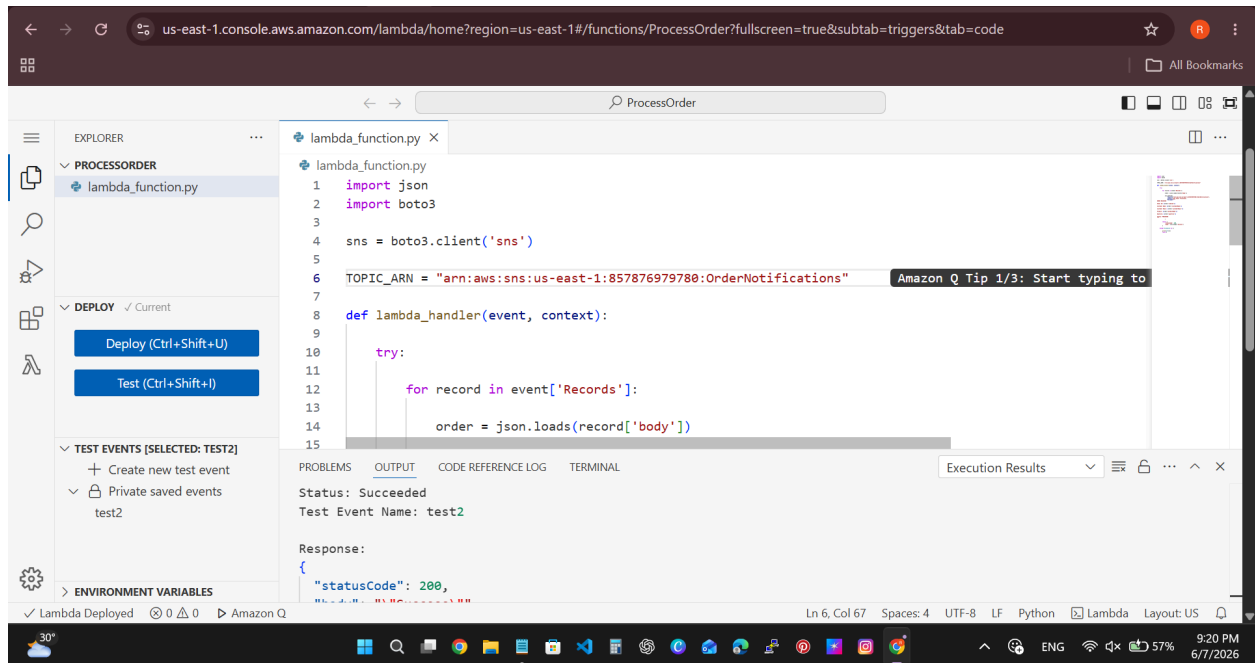
10.Lambda Function 2 – ProcessOrder

Lambda Function 2 is automatically triggered whenever a new message arrives in Amazon SQS.

Responsibilities:

- Read order details from SQS
- Process the order
- Publish a notification message to Amazon SNS

This demonstrates event-driven processing using AWS services.



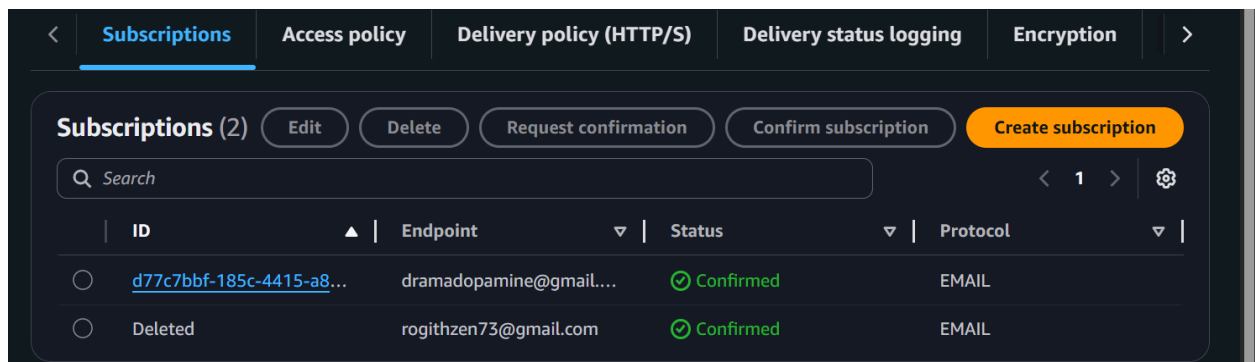
11. Amazon SNS Notification Service

Amazon SNS is used to send email notifications after successful order processing.

Notification Includes:

- Order ID
- Customer Name
- Product Name
- Quantity
- Order Status

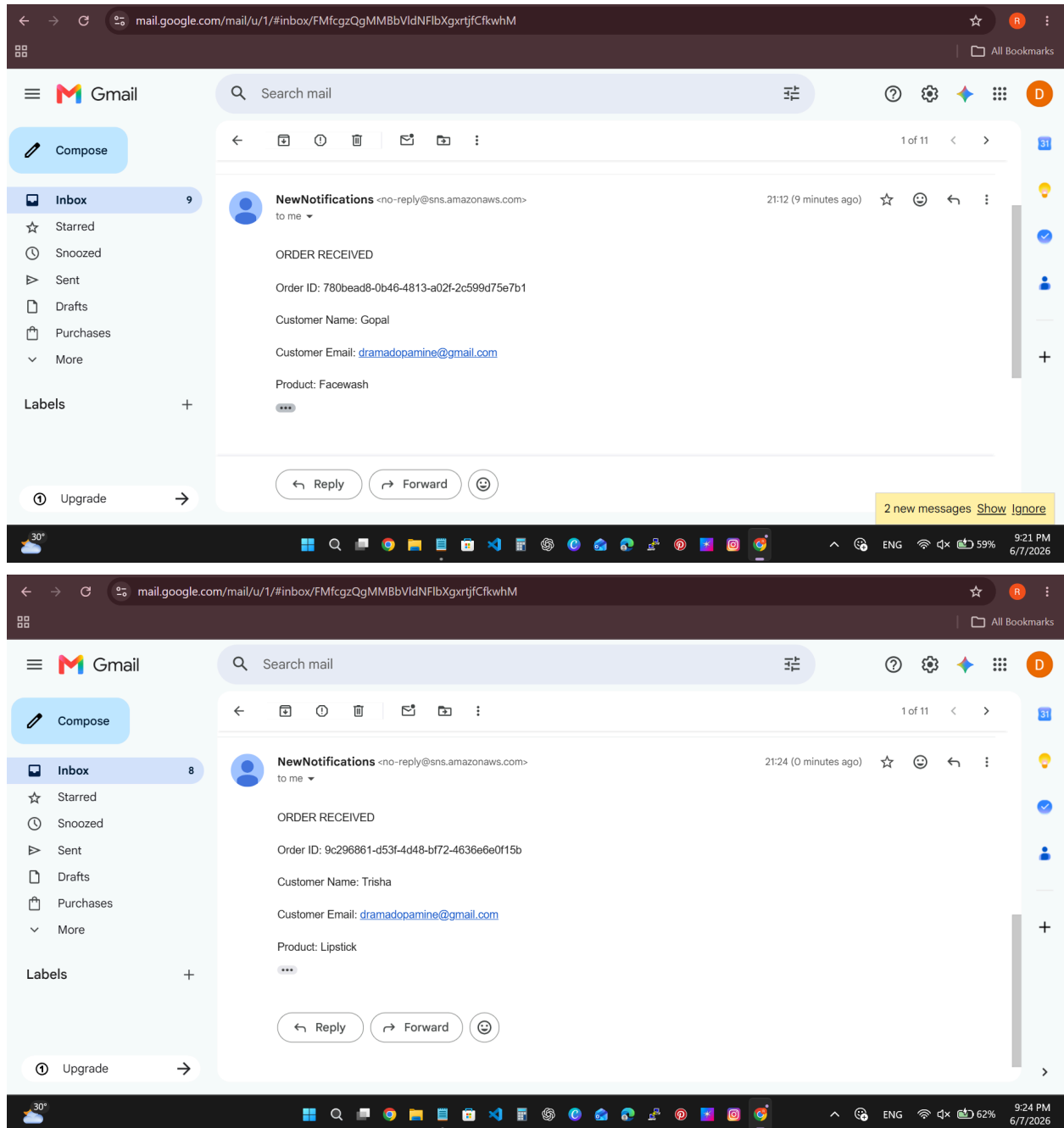
SNS allows automated communication with users without manual intervention.



12.Email Notification

Once the order is processed successfully, an email notification is sent to the subscribed email address.

The notification confirms that the order has been received and processed.



13.Challenges Encountered

API Gateway CORS Configuration

Issue:

The frontend application was unable to communicate with API Gateway due to browser security restrictions.

Resolution:

Enabled CORS in API Gateway and redeployed the API.

SNS Subscription Issue

Issue:

Email subscriptions were automatically deactivated.

Resolution:

Created and confirmed a new SNS email subscription.

API Endpoint Configuration

Issue:

Incorrect API endpoint configuration caused request failures.

Resolution:

Updated the frontend API URL with the correct API Gateway endpoint.

14.Key Outcomes

- Developed a complete serverless application using AWS.
- Implemented an event-driven architecture using Amazon SQS.
- Integrated six AWS services into a single workflow.
- Automated email notifications using Amazon SNS.
- Eliminated the need for server management.
- Improved scalability and reliability through serverless design.

15.Skills Demonstrated

Cloud Computing

- Amazon Web Services (AWS)
- Serverless Architecture
- Event-Driven Design
- Cloud-Native Development

AWS Services

- Amazon S3
- Amazon API Gateway
- AWS Lambda
- Amazon DynamoDB
- Amazon SQS
- Amazon SNS

Programming & Development

- HTML
- CSS
- JavaScript
- Python
- Boto3 SDK

16.Conclusion

This project successfully demonstrates the implementation of a Serverless E-Commerce Order Processing System using AWS cloud services. Orders are collected through a web interface, stored in Amazon DynamoDB, processed asynchronously using Amazon SQS, and email notifications are delivered through Amazon SNS.

The project showcases practical experience with serverless computing, event-driven architecture, REST API development, and cloud-native application design. It demonstrates the ability to build scalable and reliable applications using modern AWS services and industry best practices.